

AI ENGINEER ASSESSMENT SUBMISSION

ASHUTOSH SAXENA

College: BMSITM | Target Role: AI Engineer

PROJECT: AI-POWERED CREDIT RISK INTELLIGENCE PLATFORM

An End-to-End Explainable Underwriting, Machine Learning Risk Scoring, & Conversational LLaMA3 SQL Engine

BUILT FOR PORTABILITY AND TRANSPARENCY • DOCKERIZED ENVIRONMENT • INTERACTIVE STREAMLIT UI

1. Executive Summary & Banking Business Context

The Pressures on Modern Underwriting

Retail and commercial banks face immense regulatory and operational pressure to expedite credit approvals while maintaining high precision. Opaque credit scoring models increase default risks and violate auditing frameworks.

Core Platform Mission

This platform bridges the gap between sophisticated Machine Learning models and deterministic underwriting credit policies. It ensures that every automated credit decision is backed by an explicit, auditable, and explainable logic.

Key Banking Needs Addressed:

- **Identify High-Risk Borrowers:** Flag default risks early during application ingestion to protect credit portfolios.
- **Auto-Decision Support:** Scale operational bandwidth by automated approvals for creditworthy applicants and rejections for high-risk cohorts.
- **Audit and Compliance:** Satisfy federal audit criteria (e.g. Basel III, FCRA regulations) with clear decision rules.
- **Talk-to-Data Conversational NLP:** Enable non-technical credit analysts to explore databases using natural language.

2. Modular Platform Architecture

The platform is engineered around four decoupled modular layers running inside a containerized sandbox environment, ensuring high maintainability, portability, and clean interfaces.

Core Platform Layer	Technology Stack	System Mandate & Operations
1. Data & DB Layer	SQLite + Pandas	Maintains a local, self-contained relational database populated with 10,000 real dataset rows. Enforces read-only database connections for analyst operations.
2. Machine Learning Pipeline	Scikit-Learn + joblib	Implements preprocessing ColumnTransformers and class-weighted Random Forest classifiers to compute 0-100% applicant default probabilities.
3. Policy Rules Engine	Python policy objects	Applies deterministic credit underwriting knockouts (e.g. bureau scores, DTI thresholds) on top of model predictions to yield auditable decisions.
4. Conversational NLP Layer	Groq Cloud + LLaMA3 API	Translates natural language questions to SQL, runs them on SQLite, and synthesizes executive business insights in under 100ms.
5. Streamlit User Interface	Streamlit + Plotly	Provides interactive analytical dashboards, scoring sandboxes, Z-score XAI charts, and live database query terminals.

3. Exploratory Data Analysis: Demographic Risk Factors

Higher Vulnerability Among Younger Demographics

Exploratory analysis of applicant age (converted from negative DAYS_BIRTH values) reveals a strong inverse correlation with credit default. Applicants under the age of 25 demonstrate a default rate exceeding **12%**, which is 2.5x higher than applicants aged 55+.

Education Level as a Key Risk Proxy

The dataset shows a clear trend of credit repayment linked to educational backgrounds. Applicants with Lower Secondary degrees exhibit default rates of **13.7%**, whereas Higher Education graduates maintain a low **4.8%** default rate.

Demographic Underwriting Enforcements:

1. **Thin-File Youth Policy (Rule PL-01):** Applicants under 23 years old without prior credit bureau records are auto-referred for manual review to verify income stability.
2. **Co-Signer Mandates:** Enforces secondary co-signer requirements for high-risk demographic brackets scoring below prime rating thresholds.
3. **Education Variable Weight:** Categorical preprocessing encodes applicant academic achievements, which the ML pipeline heavily weights.

4. Exploratory Data Analysis: Financial & Bureau Risks

Extreme Class Imbalance

The Home Credit dataset is heavily skewed: **91.9%** of historical applicants successfully repaid, while **8.1%** defaulted. Standard ML models trained without balancing techniques predict all repayments successfully but completely miss defaults.

Debt-to-Income (DTI) Outliers

Applicants whose monthly loan annuity payments exceed **20%** of their monthly total income (DTI ratio > 0.20) present exponential default risk.

Financial & Bureau Underwriting Rules:

- **Class Weighting:** Enforces class balancing weights in ML models to prioritize learning from the minority defaulted cohort.
- **Bureau Ratings Knockout (Rule CR-01):** Substandard credit scores (``EXT_SOURCE_2 < 0.22`` or ``EXT_SOURCE_3 < 0.18``) trigger auto-rejections regardless of income.
- **Affordability Ceiling (Rule AF-01):** Imposes a hard credit policy cap, auto-rejecting applicants with a DTI ratio exceeding 45%.

5. Preprocessing & Feature Engineering Pipeline

An automated, scikit-learn compliant preprocessor pipeline converts noisy demographic inputs into a clean, normalized feature matrix for downstream model scoring.

Pipeline Stage	Operational Function	Tech/Transformation Enforced
1. Feature Selection	Filters identity columns (`SK_ID_CURR`) and separates the `TARGET` vector prior to preprocessing fits.	Drop lists & indexing
2. Numeric Imputation	Fills missing numerical columns (such as external credit bureau scores EXT_1, 2, 3) using median values to ensure zero null errors.	<code>`SimpleImputer(strategy='median')`</code>
3. Scale Standardization	Standardizes all numerical inputs to a mean of 0 and unit variance, stabilizing the classifier weights.	<code>`StandardScaler()`</code>
4. Categorical Processing	Imputes missing categorical variables with the most frequent value, then encodes them using one-hot mapping.	<code>`OneHotEncoder(handle_unknown='ignore')`</code>
5. Outlier Treatment	Caps extreme anomalies, such as the synthetic 365,243 days employed value (representing retired applicants) to zero.	Custom anomaly filters

6. Machine Learning Layer & Model Rationale

Model Selection Rationale

I trained a **Random Forest Classifier** with `n_estimators=200`` and `max_depth=10``. This model provides optimal performance for credit risks compared to boosting techniques (which are highly prone to overfitting on credit noise) and yields robust estimators without complex hyperparameter search grids.

Balanced Loss Framework

Enforces `class_weight='balanced'`` to scale loss functions relative to label frequencies. This improves the model's sensitivity to default occurrences, elevating both the ROC-AUC and PR-AUC indexes.

Model Performance Metrics

Validation Metric	Score Value
Training Accuracy	72.87%
Validation Accuracy	71.94%
ROC-AUC Score	~0.74
PR-AUC (Avg Precision)	~0.28

These scores align directly with industry benchmarks on the Kaggle Home Credit dataset, capturing defaulted debt while maintaining high operational throughput.

7. Explainable AI: The Custom Z-Score Explainer

The Docker Compilation SHAP / LIME Problem

Standard feature explainer libraries like SHAP and LIME require complex, platform-specific C++ binary compilation upon package installation. When compiling inside lightweight Docker containers across different host OS platforms (macOS/Windows), this consistently triggers compilation failures.

Features of my Custom explainer:

- **Zero Dependency:** Runs purely in native Python/Pandas without complex C++ binary compiling.
- **Ultra-Fast Performance:** Computes feature contributions in under **5ms** (vs. 2+ seconds for SHAP kernel explainers).
- **Auditable Insights:** Maps results to positive/negative influence bar charts, satisfying banking audit regulations.

Z-Score Mathematical Explainer Logic:

1. **Baseline Deviation:** Extracts individual client feature metrics and subtracts the demographic baseline average:

$$\text{Deviation} = \text{ClientValue} - \text{PopulationMean}$$

2. **Z-Score Standardizing:** Divides deviations by the demographic standard deviation, calculating exact deviation levels:

$$\text{Z_Score} = \text{Deviation} / \text{PopulationStd}$$

3. **Direction Scaling:** Weights deviations relative to a risk-importance direction vector (+1 adds risk, -1 reduces risk).

$$\text{Contribution} = \text{Z_Score} * \text{RiskDirection} * 1.5$$

8. Talk-to-Data System: Conversational SQL Engine

Why I Selected Groq LLaMA3-8B

Groq Cloud's low-latency inference engine utilizing `llama3-8b-8192` delivers sub-100ms response speeds. This offers an interactive chat experience for non-technical bank risk officers to query complex databases in plain English without writing a single line of SQL.

Double-Pass Synthesis Pipeline

1. **NLP-to-SQL Translation:** Groq maps natural English into optimized SQLite SELECT statements using direct database metadata prompts.
2. **Executive Synthesis:** Runs queries on local SQLite, and feeds tabular rows back to Groq to synthesize executive summaries under 3 sentences.

Dynamic Prompt Injectors:

To eliminate AI hallucinations, the Groq client is fed a complete database schema catalog:

- **Schema definitions:** Explicit column names and types (e.g. `SK\_ID\_CURR` as INTEGER, `TARGET` as binary index).
- **Age adjustments:** Prompt instructions mandate using `ABS(DAYS\_BIRTH) / 365.25 > X` to adjust for negative day representations in age queries.
- **Read-Only limits:** Restricts Groq output to standard SQLite SELECT statements, shielding operational DB layers.

9. Talk-to-Data Security Guardrails & Optimization

SQL Injection Interception Shield

Allowing LLMs to execute dynamic code on relational databases introduces severe security exploits. An attacker could bypass limits to read confidential credentials or inject destructive commands.

Action Blacklist Interception

My custom query runner implements a strict regex interception filter. Prior to SQLite compilation, all generated query strings are converted to uppercase and screened against a destructive keyword blacklist. If keywords like `DROP`, `INSERT`, `DELETE`, `UPDATE`, `CREATE`, or `ALTER` are found, execution is blocked instantly.

Token & Latency Optimizations:

- **Compact Matrix Formatting:** SQLite output tables are serialized using Pandas `.to_string(index=False)` to remove index columns and unnecessary characters, compressing context window tokens.
- **Low-Temperature Constraints:** Translation completions are forced at `temperature=0.0` for rigid SQLite syntaxes, eliminating hallucinatory outcomes.
- **Sub-100ms Execution:** Bypasses heavy ORM networks by routing query strings directly through SQLite connections.

10. Hybrid Credit Underwriting Rules Engine

To ensure full compliance under federal lending guidelines, model probabilities are coupled with deterministic underwriting guardrails, generating explicit audit-compliant outcomes.

Rule ID	Enforced Regulatory Underwriting Logic	Execution Outcome
Rule CR-01	Credit bureau knockout: auto-decline applicants if EXT_SOURCE_2 < 0.22 or EXT_SOURCE_3 < 0.18.	AUTO REJECT
Rule AF-01	Affordability Cap: auto-decline if monthly annuity payments exceed 45% of total income (DTI > 0.45).	AUTO REJECT
Rule ML-01	Max Risk Score Limit: auto-decline if model probability scores exceed 40%.	AUTO REJECT
Rule ML-02	Elevated Alert Limit: refer applicant to human underwriter if model scores are between 15% and 40%.	MANUAL REFER
Rule PL-01	Youth Thin Credit File: manual refer if applicant is aged < 23 years and has substandard bureau scores.	MANUAL REFER
Rule SP-01	Super Prime Track: auto-approve if external bureau scores are >= 0.70, DTI <= 15%, and ML risk < 5%.	AUTO APPROVE

11. Production Operations & Dockerized Setup

Lightweight Multi-Stage Docker Containerization

I engineered a multi-stage Docker build to keep my production runtime environment extremely streamlined and secure. Built dependencies and temporary compiler files are discarded in the builder stage, and only compiled libraries are copied to the runner stage. Binds container services to port `8501` to expose Streamlit to host layers.

Relational SQLite Volume Persistence

Avoids massive cloud infrastructure database costs by mounting SQLite database files (`data/credit\_risk.db`) via host volumes. User activity and underwriting logs persist across container cycles.

Step-by-Step Submission Deployment Guide:

1. Set Environment Variables:

Copy `.env.example` to `.env` to load configurations:

```
cp .env.example .env
```

2. Build and Run the Containers:

Orchestrate the app, volumes, and API parameters in a single command:

```
docker-compose up --build
```

3. Access the Streamlit Dashboard:

Open browser at: <http://localhost:8501>

4. Local Development Fallback:

Run locally without Docker if needed:

```
pip install -r requirements.txt
```

```
streamlit run app.py
```